Fast Gradient Boosting without Trees for Large-Scale Time Series Classification

Matthew Yang
Student ID 36196479

October 19, 2026

Abstract

On large time series datasets, training the classifier becomes increasingly expensive. A boosted tree ensemble such as XGBoost searches every feature for splits in every round, so its cost grows with the size of the data, and it must hold the whole training set in memory. We replace the tree with a *global partition*: a few threshold predicates, placed between pairs of hard, differently labelled examples, that index a table of leaf values. The resulting booster, HashBoost, trains from mini-batches on the GPU. HashBoost's cost is set by the round budget rather than by the size of the data, and its memory does not grow with the training set. We evaluate this model on MONSTER, a benchmark of 26 large time series datasets, using QUANT features. With both models tuned by the same procedure and scored on the official test folds, HashBoost matches tuned XGBoost's accuracy and trains 8.2× faster in total on the same GPU: up to 31× faster on the larger datasets where XGBoost is most expensive, though slower on the cheapest. On the small datasets of the UCR archive the advantage disappears: QUANT's own ExtraTrees classifier is both faster and more accurate there.

1 Introduction

Gradient boosted decision trees [1], in implementations such as XGBoost [2], LightGBM [3] and CatBoost [4], are the default choice for tabular prediction, where they remain competitive with, and frequently superior to, deep learning [5, 6]. Tree ensembles are also the usual classifier on top of interval-based time series features: QUANT [7] passes its quantile features to extremely randomised trees, and PULSAR [8] is used the same way.

At the scale of the MONSTER benchmark [9], with up to 5.9 million series per dataset and tens of thousands of features after QUANT, the classifier becomes the expensive part. To grow a boosted tree, the learner searches for split points on every feature, and a multiclass booster grows one tree per class per round. Each round therefore costs time in proportion to rows, features and classes together. Current techniques such as histogram binning, approximate split finding, gradient-based sampling and feature bundling [2, 3, 10] reduce the constant factor, but every round must still revisit the whole training set and hold it in memory.

This paper replaces each tree with a *global partition* of the feature space. We use b axis-aligned threshold predicates, applied to every example at once, whose b bits index a table of 2^b leaf values. A predicate takes a random feature and puts its threshold at the midpoint of two hard, differently labelled examples, so there is no expensive search over the training set. Because the partition does not depend on which examples are in the current batch, its leaf values can be re-estimated from any later mini-batch, and the model never needs the training set all in memory at once.

Our claim is that this booster matches tuned XGBoost's accuracy on large-scale time series classification, and that because its cost does not grow with the data, it trains far faster exactly where XGBoost is most expensive. We ask:

RQ1 Given the same tuning effort, does HashBoost match a tuned boosted tree ensemble's accuracy on MONSTER's test folds?

RQ2 How much faster is HashBoost, on which datasets, and why? What does each model reach in the same training time, and what memory does each need?

RQ3 Where does the advantage stop? Does it hold on small training sets?

Contributions.

1. **HashBoost**, a gradient booster whose weak learner is a global partition, with a GPU implementation that trains from mini-batches (Section 3).
2. **A test-fold comparison on all 26 MONSTER datasets** against XGBoost tuned by the same procedure and against QUANT's ExtraTrees, trained on whole pools of up to 4.65 million rows (Section 5.1).
3. **An account of where the time goes**: per-dataset speedups explained by XGBoost's cost per round, a time-matched comparison, and memory against training-set size (Section 5.2).
4. **The small-data boundary** on the 112 UCR datasets (Section 5.3).

Our code is available at `https://github.com/minip8/FIT2082`.

2 Background and Related Work

Boosting. Boosting combines weak learners into a strong one, as formalised by AdaBoost [11] and generalised to differentiable losses as gradient boosting [1]. Trees are the conventional weak learner, but nothing in the framework requires them. Modern implementations fit each tree to a second-order expansion of the loss, so that a leaf's value is a Newton step $-\sum g / \sum h$ over its examples [2]. HashBoost keeps this leaf rule and changes how examples are assigned to leaves.

Oblivious trees and decision tables. CatBoost [4] grows *oblivious* trees, in which every node at a given depth uses the same split, so a depth-d tree is d predicates indexing 2^d leaves. This is the structure of one HashBoost round, and neural oblivious ensembles [12] use it too. CatBoost chooses each level's split greedily by gain over the whole training set and fixes the leaves once the tree is built. Boosted decision tables [13] are the closest relative. They choose a table's tests greedily by gain over the whole training set, then *backfit* them, re-choosing each test given the others. Their leaves are set once, and earlier tables are never revisited. HashBoost does the opposite on both counts. Its tests are not searched for over the training set, and every earlier round's leaves are re-estimated by each later batch. That is closer to backfitting additive models [14], applied online.

Randomised partitions. Random forests [15] and extremely randomised trees [16] showed that much of a tree's split search can be replaced by randomness, provided the ensemble is large. A random global partition can also be read as a locality-sensitive hash [17]: examples that share a code agree on b random comparisons. HashBoost's thresholds are random but not uninformed. They sit between two hard examples of different classes, so each bit tends to separate examples the current model confuses.

Scaling and streaming. The usual way to make tree boosting fast at scale is to keep the search and accelerate it. GPU histogram construction [18] builds each tree's histograms in parallel, and out-of-core training [2] pages the binned matrix from disk. Both still pass over the whole training set every round. Learning from a stream is the other route. Hoeffding trees [19] grow one tree from a stream, splitting a leaf once enough examples have arrived to choose its test with confidence, and online boosting [20, 21] updates every weak learner as each example arrives. HashBoost is closest to online boosting: it updates all existing learners from each new batch. Unlike online boosting, its learners' structure is fixed when they are created, so an update is a scatter-add rather than a refit.

Time series classification. Random-feature methods have shown that data-dependent search can often be replaced by cheap random transformations: ROCKET [22], MiniRocket [23] and Hydra [24] pair random convolutional kernels with a linear classifier, at a fraction of the cost of ensembles such as HIVE-COTE [25]. Interval methods (Time Series Forest [26], CIF [27], DrCIF [25], rSTSF [28], QUANT [7], PULSAR [8]) all end in some form of tree. We keep QUANT as a fixed transform and replace only the classifier.

3 Method

3.1 A round is a global partition

Let $x \in \mathbb{R}^p$ be a feature vector and k the number of classes. A round r is defined by b pairs $(j_{r,i}, t_{r,i})$ of a feature index and a threshold, for $i = 0, \ldots, b-1$. It maps x to a b-bit code

$$c_r(x) = \sum_{i=0}^{b-1} 2^i \mathbb{1}\big[x_{j_{r,i}} \le t_{r,i}\big] \in \{0, \ldots, 2^b - 1\}, \tag{1}$$

and has a table $L_r \in \mathbb{R}^{2^b \times k}$ of leaf values. The model after R rounds is

$$F(x) = \sum_{r=1}^{R} L_r[c_r(x)] \in \mathbb{R}^k, \tag{2}$$

and predicts $\mathrm{softmax}(F(x))$. A round is an oblivious tree of depth b whose splits are fixed when it is created. The default is $b = 8$, so a round has 256 buckets.

3.2 Choosing the predicates

Given the current batch, its labels and the model's predicted probabilities, a new round's predicates are chosen as follows:

1. Order the batch by cross entropy, hardest first.
2. Walk that order and greedily form b pairs of examples with different labels.
3. For each pair (u, v), draw a feature j uniformly at random and set $t = (u_j + v_j)/2$.

Each bit therefore separates two examples the model currently gets wrong, and the only search is the sort in the first step. A *gain-selected* variant draws a pool of bK such hard-pair candidates and fills the bits one at a time, the way CatBoost grows an oblivious tree level by level (Section 2): each bit takes the unused candidate with the highest second-order gain, $\sum G^2/(H + \lambda)$ with $\lambda = 1$, summed over classes and over the cells the earlier bits have made. Here G and H are the batch's gradient and hessian summed within a cell. Unlike CatBoost, it scores only these bK random candidates, on one batch, rather than every feature over the whole training set.

Algorithm 1 One HashBoost step on a mini-batch (X, y) of n examples, adding M rounds.

for $m = 1, \dots, M$ **do**
  $F \leftarrow \sum_{r=1}^{R} L_r[c_r(X)]$ ▷ (n, k) logits
  $p \leftarrow \mathrm{softmax}(F)$; $g \leftarrow p - \mathrm{onehot}(y)$; $h \leftarrow p(1-p) + 10^{-3}$
  choose predicates of round $R+1$ from hard pairs in (X, y, p)
  for $r = 1, \dots, R+1$ **do** ▷ all rounds, including the new one
    $G_r[c_r(X)] \mathrel{+}= -g$; $H_r[c_r(X)] \mathrel{+}= h$ ▷ scatter-add
    $L_r \leftarrow \eta\, G_r / (H_r + \epsilon)$
  end for
  $R \leftarrow R + 1$
end for

3.3 Fitting the leaves

We use the softmax cross-entropy gradient $g = p - \mathrm{onehot}(y)$ and a floored diagonal hessian $h = p(1-p) + 10^{-3}$. Every round keeps running sums $G_r, H_r \in \mathbb{R}^{2^b \times k}$ of $-g$ and h over the examples that landed in each bucket, and its leaf values are

$$L_r[s] = \eta \, \frac{G_r[s]}{H_r[s] + \epsilon}, \tag{3}$$

with learning rate $\eta = 0.1$. By default $\epsilon = 10^{-6}$; a larger ϵ is an L2 penalty that shrinks thinly populated leaves.

The key departure from tree boosting is *when* these sums are accumulated. A tree's leaves are fitted once and never change. In HashBoost, **every mini-batch is accumulated into every existing round**, with gradients evaluated at the current model (Algorithm 1). A round created early keeps absorbing evidence from every later batch, so it is not tied to the batch that created it, and the model can learn from a stream. Adding $M > 1$ rounds per batch raises capacity without more passes over the data.

3.4 What training costs

A batch of n rows costs encoding and scatter-adds over the rounds it touches, $O(nRb + nRk)$, plus choosing its new predicates on that batch alone. Nothing in this depends on the size N of the training pool. Total cost is the number of batches times the cost per batch, and memory holds only one batch and the tables. A boosted tree's round instead builds feature histograms over all N rows, for each of the k class trees, so its cost per round grows with $N \cdot p \cdot k$, and it needs the binned matrix of all $N \times p$ values.

HashBoost has a scaling problem of its own. Because every batch updates every round, a batch costs $O(R)$ once there are R rounds, so T batches cost $O(T^2)$. *Round freezing* removes this growth. With a window of W rounds, only the newest W keep accumulating, and a frozen round's contribution to an example never changes again. Given stable row ids, each row's frozen contribution is summed once and cached in an (N, k) table. A batch then costs $O(W + f)$, where f is the number of rounds frozen since its rows were last seen, and total cost becomes linear in rounds.

3.5 Implementation

HashBoost runs entirely on the GPU in PyTorch. The first implementation was a CPU prototype in numba, which we kept as a test oracle for the GPU version: given the same predicates, the two agree to 4×10^{-7} in logits with identical predictions. The GPU version trains Pedestrian's default 800 rounds in about 7.5 s, against 101.5 s for the prototype.

The code is organised so that an experiment can change one component at a time. The predicate family, the split selector and the loss are separate interfaces (Python `Protocol`s), and every experimental change is an opt-in option, so the default model stays the prototype's algorithm. On the GPU the greedy pairing of hard examples runs as a single Triton kernel, so a round needs no transfer back to the host, and a test checks that it chooses the same pairs as the Python version. The 224 tests also cover the QUANT port against its upstream reference. Every run writes its results to a file named after its commit, so every number here can be traced to the code that produced it.

4 Experimental Setup

MONSTER. We use all 26 classification datasets in MONSTER [9] (Table 1), with QUANT [7] features (depth 6, div = 4, our PyTorch port). MONSTER supplies five fixed test folds. For each fold, the *pool* is every series outside that test fold. A fixed shuffle of the pool (seed 42) gives a validation slice and a tune slice of 4,096 rows each, or a fifth of the pool each when it has fewer than 73,728 rows, and every model trains on all the rest. The tune slice is used for every choice made after training, such as early stopping. Validation error is used only to choose configurations.

Configurations. HashBoost and XGBoost were tuned the same way, on fold 0's validation error alone. We screened one change at a time on 10 of the 26 datasets (CrowdSourced, DREAMERA, FordChallenge, InsectSound, LenDB, PAMAP2, STEW, Traffic, UCIActivity

Table 1: The 26 MONSTER datasets. "Pool" is the number of series outside test fold 0, and "train" the rows trained on after the validation and tune slices are taken from it. Classes are those present in the labels.

dataset	series	pool	train	classes	QUANT features
AudioMNIST-DS	30,000	24,000	15,808	10	35,349
CornellWhaleChallenge	30,000	24,000	15,808	2	35,349
CrowdSourced	12,289	9,456	5,674	2	31,542
DREAMERA	170,246	140,638	132,446	2	31,542
DREAMERV	170,246	140,638	132,446	2	31,542
FordChallenge	36,257	29,003	20,811	2	14,130
FruitFlies	34,518	27,614	19,422	3	44,031
InsectSound	50,000	40,000	32,768	10	5,470
LakeIce	129,280	103,424	95,232	3	1,652
LenDB	1,244,942	983,483	975,291	2	14,940
MosquitoSound	279,566	223,652	215,460	6	33,178
Opportunity	17,386	12,184	7,312	5	117,746
PAMAP2	38,856	28,961	20,769	12	54,184
Pedestrian	189,621	151,696	143,504	82	212
S2Agri-10pc-17	5,850,881	4,661,858	4,653,666	17	2,120
S2Agri-10pc-34	5,850,881	4,661,858	4,653,666	29	2,120
STEW	28,512	21,384	13,192	2	31,542
Skoda	14,117	11,290	6,774	11	62,520
TimeSen2Crop	1,135,511	839,971	831,779	15	30,096
Tiselac	99,687	79,907	71,715	9	2,040
Traffic	1,460,968	1,168,774	1,160,582	7	212
UCIActivity	10,299	7,898	4,740	6	10,440
USCActivity	56,228	43,060	34,868	12	6,252
WISDM	17,166	11,989	7,195	6	3,126
WISDM2	149,034	98,094	89,902	6	3,126
WhaleSounds	105,163	84,130	75,938	8	22,105

and WISDM), fixed two configurations per model, and ran both on all 26. For each dataset and model the configuration with the lower fold-0 tune error was frozen. Each frozen configuration was then trained once on each of the five folds and scored once on that fold's test series, at the round its tune error chose. No test series was seen before this.

Both HashBoost configurations train for up to 5,000 batches of 4,096 rows, evaluate every 50 batches, stop once their best tune evaluation is 1,000 batches old, and freeze rounds older than 128 batches:

- `G`: 6-bit gain-selected rounds ($K = 64$, so a pool of 384 candidates), $\epsilon = 100$, 8 rounds per batch. Picked on 18 datasets.
- `L`: 8-bit hard-pair rounds, $\epsilon = 1$, 16 rounds per batch. Picked on the largest pools (LenDB, both S2Agri, Traffic, Pedestrian), WhaleSounds and the EEG sets DREAMERA and DREAMERV.

Gain selection had not helped on 65,536-row slices, but on whole pools, with 6-bit rounds and a heavy leaf penalty, it was the largest single gain in the screen (CrowdSourced's validation error fell from 0.042 to 0.006). Over five model seeds on fold 0, the same configuration was picked every time on 25 of the 26 datasets.

We compare against one tree booster, tuned with the same effort as HashBoost: XGBoost, because it trains on the same GPU, so the times compare directly, and can train out of core (Section 5.2). On early 65,536-row slices, LightGBM's validation error was in the same range. XGBoost's screen raised its cap to 5,000 rounds with 100 rounds of patience, then varied depth, learning rate, subsampling, L2 regularisation and the number of bins. Its configurations are `d4` (depth 4, learning rate 0.1; picked on 17 datasets) and `d8` (depth 8, learning rate 0.05, 64 bins; picked on 9). The third model, ExtraTrees with QUANT's own settings (200 trees, entropy criterion, a tenth of the features per split), chooses nothing after training, so it trains on the same rows and needs no tune slice.

UCR. For the small-data boundary we use the 112 univariate UCR datasets [29] that the bake-off reports on [30]: the archive's 128, less the 15 with missing values or unequal lengths and Fungi, which has one training series per class. Each is used on its default train/test split. The training sets have 16 to 8,926 series (median 190). There is no validation split, so every model runs at fixed settings and we report test error: HashBoost at 800 rounds of 8 bits, ExtraTrees with QUANT's settings, and XGBoost at its library defaults (100 rounds, learning rate 0.3, depth 6).

Hardware and timing. Every HashBoost and XGBoost run on MONSTER had one NVIDIA L40S (46 GB) on Monash's M3 cluster to itself, so their times are directly comparable. Extra-Trees ran on CPU nodes with 36 cores, so its time is not comparable. Training time includes the QUANT transform for both boosters. UCR and the streaming runs in Section 5.2 used one RTX 4060 (8 GB). Seeding does not make HashBoost reproducible, because float32 scatter-adds on the GPU accumulate in a nondeterministic order. On Pedestrian the run-to-run standard deviation of validation error is about 0.003.

5 Results

5.1 Accuracy at scale (RQ1)

Table 2 gives each model's test error on the 26 MONSTER datasets, averaged over the five test folds, with training times, and Table 3 summarises the pairwise comparisons. As a check, every model, reloaded for testing, reproduced the validation error it had recorded at its tune-chosen round, so test error was scored at the intended point.

Table 2: MONSTER: test error, mean of the five test folds, one run per fold. HashBoost and XGBoost use the configuration each picked on fold 0. Times are training seconds summed over the five folds, QUANT included: HashBoost and XGBoost on one L40S, ExtraTrees on 36 CPU cores. The lowest error in each row is bold. †One fold rerun with a fixed intercept (footnote 1).

	HashBoost		XGBoost		ExtraTrees	time (s)		
dataset	config	error	config	error	error	HB	XGB	ET
AudioMNIST-DS	G	**0.0816**	d4	0.0884	0.1270	503	885	429
CornellWhaleChallenge	G	0.1250	d8	**0.1233**	0.1262	290	424	412
CrowdSourced	G	0.3501	d4	0.3622	**0.3396**	192	95	119
DREAMERA	L	0.5161	d8	0.5312	**0.4867**	438	1,775	4,253
DREAMERV	L	**0.4036**	d8	0.4038	0.4179	434	2,264	4,235
FordChallenge	G	0.0823	d8	0.0731	**0.0715**	196	152	224
FruitFlies	G	**0.0458**	d4	0.0462	0.0619	383	384	670
InsectSound	G	**0.1872**	d4	0.1891	0.2268	520	357	148
LakeIce	G	**0.0018**	d4	0.0019	0.0024	195	38	116
LenDB	L	0.0538	d4	**0.0522**	0.0551	728	1,364	25,843
MosquitoSound	G	0.1052	d4	**0.1008**	0.1596	1,855	5,054	7,366
Opportunity	G	0.1322	d4	**0.1305**	0.1394	399	852	761
PAMAP2	G	**0.2143**	d4	0.2246	0.2288	277	1,025	1,013
Pedestrian	L	**0.1746**	d8	0.1772	0.2135	2,588	792	61
S2Agri-10pc-17	L	0.2829	d8	0.2525	**0.2521**	1,463	30,068	18,291
S2Agri-10pc-34	L	0.2828	d8	**0.2530**	0.2638	2,013	61,933	16,151
STEW	G	**0.1937**	d4	0.1993	0.2371	325	310	382
Skoda	G	0.0567	d4	0.0610	**0.0565**	354	784	293
TimeSen2Crop	G	0.1412	d4	**0.0849**	0.1808	1,926	35,655	32,910
Tiselac	G	**0.2139**	d4	0.2140	0.2172	396	268	115
Traffic	L	0.2841	d8	0.2939	**0.2488**	846	407	375
UCIActivity	G	**0.0521**	d4	0.0675	0.0855	169	46	21
USCActivity	G	0.2428	d4	**0.2396**	0.2751	497	305	188
WISDM	G	0.1152	d4	**0.1131**†	0.1298	161	22	17
WISDM2	G	0.2556	d4	**0.2541**	0.2614	458	206	297
WhaleSounds	L	0.3128	d8	**0.2802**	0.2936	612	3,633	1,856
total						18,214	149,099	116,546

Table 3: MONSTER: pairwise comparisons over the 26 datasets. The ratio is the geometric mean of the first model's error over the second's. W/T/L counts the datasets where the first has the lower, equal and higher five-fold mean. "Clear" counts those where a 95% paired-t interval over the five per-fold differences excludes zero, either way. p is a two-sided Wilcoxon signed-rank test on the five-fold means, Holm-adjusted over the three comparisons.

comparison	ratio	W/T/L	clear (W/L)	p
HashBoost vs XGBoost	1.016	14/0/12	1/6	0.98
HashBoost vs ExtraTrees	0.904	18/0/8	9/5	0.11
XGBoost vs ExtraTrees	0.890	20/0/6	13/3	0.031

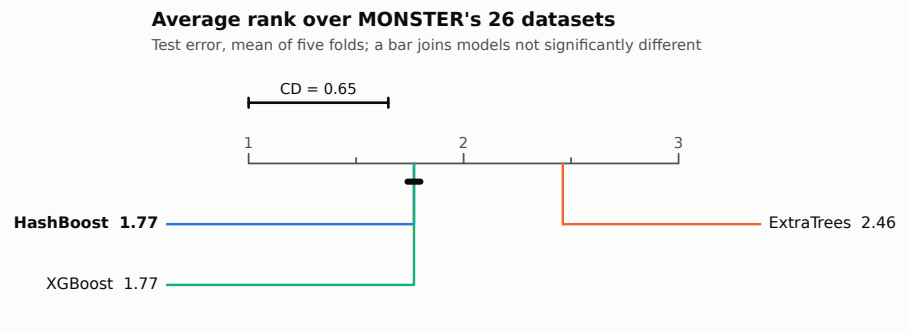


Figure 1: Critical difference diagram over MONSTER's 26 datasets [31]: test error, mean of the five folds. A Friedman test rejects equal ranks ($p = 0.016$). The bar joins models within the Nemenyi critical difference (0.65) of each other at $\alpha = 0.05$.

HashBoost matches tuned XGBoost. HashBoost's error is 1.016 times XGBoost's in geometric mean. It is lower on 14 datasets and higher on 12, and lower on 61 of the 130 dataset-folds. The Wilcoxon test finds no difference ($p = 0.98$). A non-significant test alone does not show that two models are equal, so we also bound the gap: a 95% t-interval over the 26 datasets' log error ratios gives a geometric-mean ratio between 0.97 and 1.07, so on average neither model's error is more than about 7% above the other's[1]. Where one model clearly leads, it is usually XGBoost: on 6 datasets, including both S2Agri subsets and TimeSen2Crop (where HashBoost's error is 1.66 times XGBoost's), against 1 for HashBoost (AudioMNIST-DS). HashBoost also does better on the 10 datasets it was developed on (ratio 0.973) than on the other 16 (1.045).

Both boosters rank ahead of ExtraTrees. The mean ranks are 1.77 for both boosters and 2.46 for ExtraTrees (Figure 1), 0.69 behind, just beyond the Nemenyi critical difference of 0.65. The margin is narrow for HashBoost: its pairwise Wilcoxon test against ExtraTrees is not significant ($p = 0.11$, against 0.031 for XGBoost), although its error ratio of 0.90 has a 95% interval of 0.84–0.97 that excludes 1. ExtraTrees' largest losses are on big acoustic and sensor pools: MosquitoSound (1.52 times HashBoost's error), AudioMNIST-DS, FruitFlies, Pedestrian, InsectSound and TimeSen2Crop.

Validation error is optimistic. Test error is about 3.3 times validation error for all three models, because the test folds differ in distribution from the pool that the validation and tune slices are drawn from. Every model is tuned on the same slices, so the comparison stays fair.

5.2 Where the time goes (RQ2)

The speedup follows XGBoost's cost. On the same GPU, HashBoost trained 8.2 times faster than XGBoost in total: 18,214 s against 149,099 s over the 130 dataset-folds. The total hides a wide range. Per dataset, HashBoost is faster on 14 of the 26, with a median speedup of 1.23, and the range runs from 31 times faster on S2Agri-10pc-34 to 7 times slower on WISDM. Figure 2 (left) shows what decides it. The speedup rises with $N \cdot p \cdot k$, the rows, features and classes that set XGBoost's cost per round (Spearman $\rho = 0.87$, $p = 7 \times 10^{-9}$), while HashBoost's cost is set by its batch budget. On the cheapest datasets, such as WISDM, LakeIce and UCIActivity, XGBoost finishes in seconds and HashBoost's fixed cost per round makes it the slower model. On the most expensive, S2Agri and TimeSen2Crop, XGBoost takes hours whilst HashBoost takes minutes.

[1]One XGBoost run failed: WISDM's test fold 3 holds every series of two of its six classes, and XGBoost 3.4's intercept, estimated from a pool that lacks them, left it at test error 0.683. We report a rerun with the intercept fixed at 0.5, the default before version 2.0 (0.178), with the original run's time. Counting the failed run, the ratio is 0.992 ($p = 0.71$).

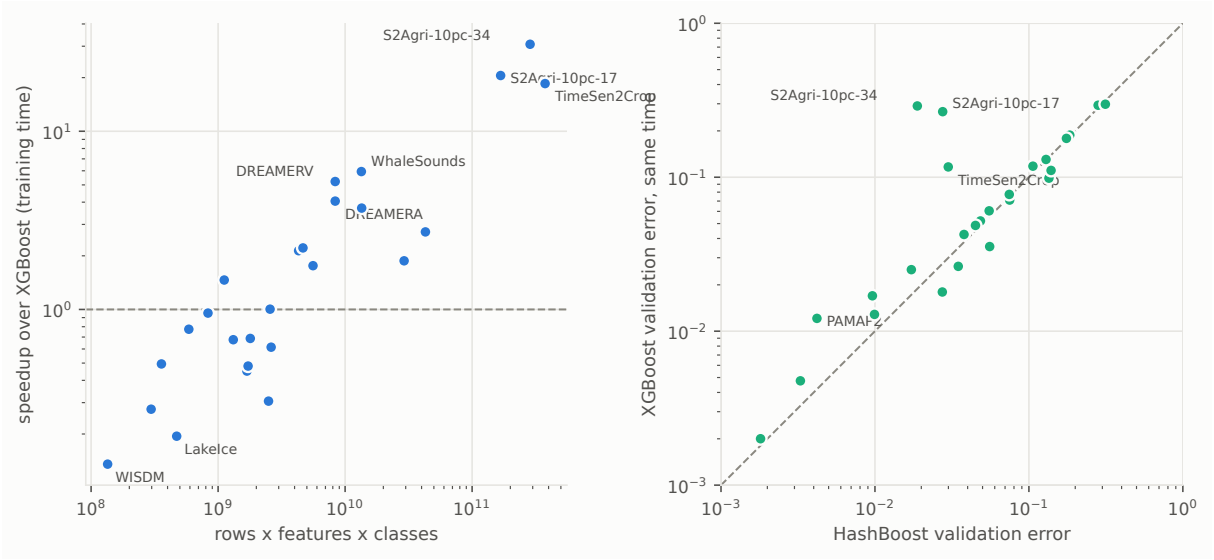


Figure 2: Left: HashBoost's training speedup over XGBoost on each MONSTER dataset (time summed over the five folds, one L40S) against rows × features × classes. Above the dashed line, HashBoost is faster. Right: validation error when XGBoost is given the same training time as HashBoost, mean of the five folds. XGBoost's round is chosen on tune error among the rounds it completed in that time. Above the diagonal, HashBoost is ahead at equal time.

At equal time, XGBoost is far less accurate on the expensive datasets. Figure 2 (right) gives XGBoost exactly HashBoost's total training time on each fold, patience included, and chooses its round on tune error among those it completed. On validation error, HashBoost is then ahead on 19 of the 26 datasets, and XGBoost's error is 1.35 times HashBoost's in geometric mean. That difference is not significant across all 26 (Wilcoxon $p = 0.24$), because on the cheaper datasets XGBoost finishes within HashBoost's time anyway. The gap is concentrated where training is expensive: in HashBoost's time, XGBoost reaches 0.27 and 0.29 on the two S2Agri subsets, against HashBoost's 0.028 and 0.019, and 0.117 on TimeSen2Crop against 0.030. Figure 3 shows the curves behind these numbers for the six largest pools.

HashBoost's budget ran out on the largest pools. Run to completion, XGBoost overtakes HashBoost on S2Agri and TimeSen2Crop, and those are among the datasets where it clearly leads on test. But HashBoost was stopped by its budget, not by its tune error: it reached its 5,000-batch cap on 14 of the 15 S2Agri and Traffic folds with its best tune error in the last 1,000 batches, still improving. On S2Agri-10pc-34 it reached its tune-chosen point in 375 s, where XGBoost needed 12,661 s. A larger budget would cost minutes, and might close part of that gap.

Memory does not grow with the training set. We streamed LenDB's whole training pool (975,291 rows: 6.3 GB as raw series and 58 GB after QUANT) through both models on the RTX 4060. HashBoost's peak GPU memory was the same, 2,572 MB, at 65,536 rows and at the full pool, because each batch is folded into the tables and freed. XGBoost's binned matrix grows linearly with the pool, to 14.6 GB (Figure 4, right). On the 8 GB card it had to page that matrix from disk every round, at 116 s per round against 0.68 s at 65,536 rows. In the MONSTER benchmark on the L40S, tuned XGBoost held LenDB's matrix on the card at 17.1–17.8 GB, against 3.4–4.9 GB for HashBoost's `L`.

Freezing keeps the cost per batch flat. Without freezing, per-batch cost grows with rounds: on a streamed Traffic run it rose from 2 ms to 14.5 ms over 2,840 rounds, while a

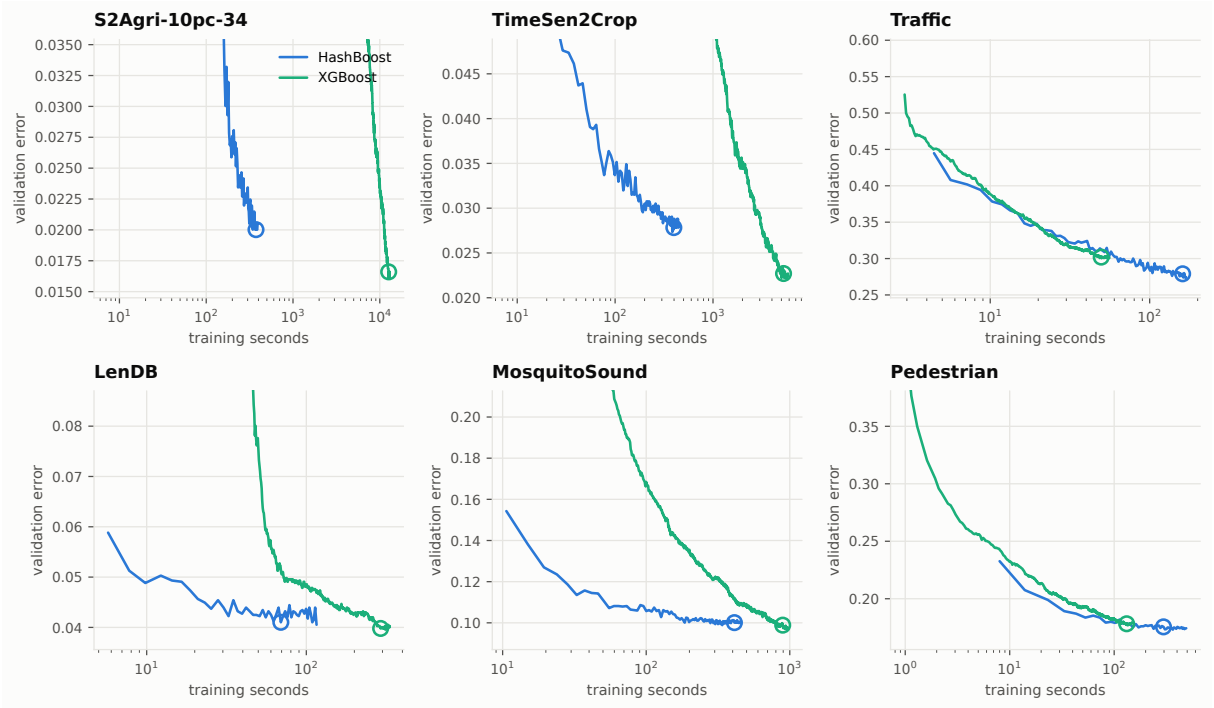


Figure 3: Validation error against training time on fold 0 for the six largest MONSTER pools, each booster at its frozen configuration, on one L40S, QUANT included. HashBoost's time is measured at each evaluation; XGBoost's is its build plus its mean time per round. Rings mark the tune-chosen point. The horizontal axis is logarithmic.

window of one or two epochs' rounds held it level (Figure 4, left). On Pedestrian, with two rounds per batch, a window of ten epochs matched the unfrozen model's accuracy in 2.4 times less time (validation error 0.2135 ± 0.0032 in 10.8 s against 0.2133 ± 0.0025 in 25.6 s, three runs each). Every MONSTER configuration freezes rounds older than 128 batches.

5.3 The small-data boundary (RQ3)

On the UCR 112 the training sets are small, and HashBoost loses both its accuracy and its speed advantage (Table 4). Our ExtraTrees reproduces the published QUANT results [30] (mean error 0.1475 against 0.1456), and it is both the most accurate model, significantly ahead of both boosters (Figure 5), and the fastest, at 70 s for all 112 against HashBoost's 183 s. At these sizes a round's cost is fixed overhead rather than arithmetic. HashBoost at its MONSTER defaults is level with default XGBoost (mean ranks 2.36 and 2.39). Its 8-bit rounds were set for buckets that see thousands of rows, and most UCR training sets cannot fill them. Resized for small data, with two bits and 3,200 rounds chosen by two-fold cross-validation over the datasets, HashBoost is significantly ahead of XGBoost (lower error on 82 datasets, higher on 29, $p = 4.5 \times 10^{-8}$) and still behind ExtraTrees (28 against 63, $p = 1.2 \times 10^{-5}$), at 7.5 times the time. Which classifier suits QUANT's features therefore depends on scale: an average of random trees on small training sets, and a booster on large ones.

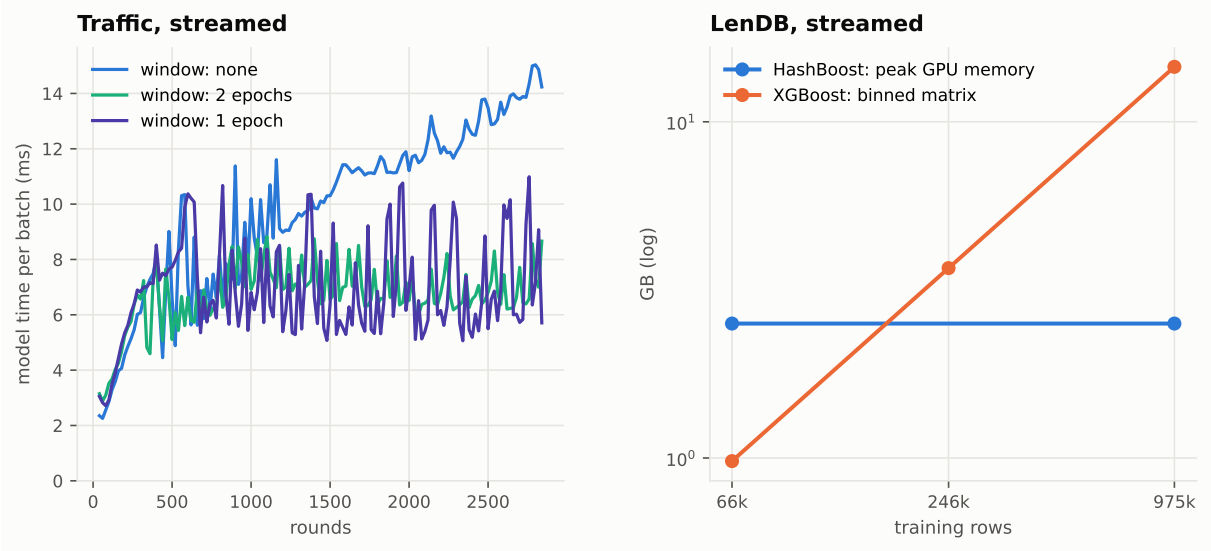


Figure 4: Left: HashBoost's model time per batch against rounds on a whole-pool Traffic stream (1.16 million rows, one round per batch, RTX 4060), unfrozen and with rounds older than one or two epochs frozen. Right: what each model needs available as LenDB's training pool grows. HashBoost's peak GPU memory is measured; XGBoost's binned matrix is measured on disk at 245,760 and 975,291 rows and derived from the same bytes per element at 65,536.

Table 4: UCR 112: test error on the default splits, one run per dataset. Mean rank is among the three models we ran. W/T/L is HashBoost's record against each row. p is a two-sided Wilcoxon signed-rank test against HashBoost, Holm-adjusted over the four comparisons. The published rows are the bake-off's resample 0, which is the default split.

model	mean error	median	mean rank	W/T/L	p	total fit
HashBoost	0.1941	0.1705	2.36	–	–	183 s
ExtraTrees	**0.1475**	0.1205	**1.25**	8/8/96	2.6×10^{-14}	70 s
XGBoost	0.2101	0.1912	2.39	58/3/51	0.082	413 s
QUANT (published)	0.1456	0.1177		13/5/94	4.9×10^{-14}	
HC2 (published)	0.1236	0.0753		8/7/97	3.8×10^{-15}	

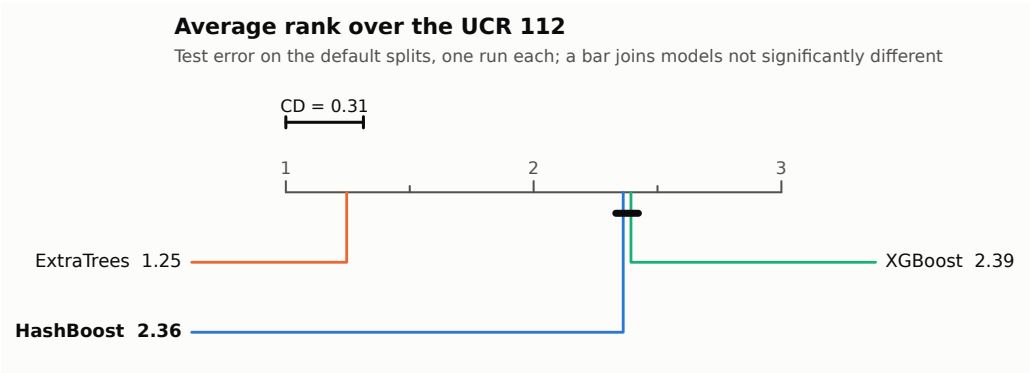


Figure 5: Critical difference diagram over the UCR 112 [31]. A Friedman test rejects equal ranks ($p = 3.6 \times 10^{-22}$). A bar joins models within the Nemenyi critical difference (0.31) of each other at $\alpha = 0.05$.

6 Discussion

Speed against accuracy. Because the two models' costs scale differently, the speedup is largest where the classifier's cost was a problem, which is where HashBoost is meant to be used. There it also gives up the most accuracy once XGBoost has run to completion (its error is 1.12 times XGBoost's on S2Agri and 1.66 times on TimeSen2Crop), while being far ahead at equal time. On S2Agri, HashBoost stopped at its batch budget while still improving, so the trade-off is at least partly set by a budget tuned on smaller pools rather than by a ceiling of the model. On TimeSen2Crop it reached the budget on only two folds of five, so the budget explains less of that gap, the largest in the benchmark.

Limitations.

- The MONSTER benchmark is one run per test fold. Over five model seeds on fold 0, HashBoost's standard deviation was 0.0002 to 0.0024 on most datasets.
- Both models were tuned for accuracy, not speed, and only on 10 of the 26 datasets, which flatter HashBoost (Section 5.1).
- The time-matched comparison is on validation error, because test error was scored only at each model's own chosen round. XGBoost's clock is its build plus its mean time per round.
- Configurations were chosen on validation error, which underestimates test error about 3.3-fold, so they may not be the best on test, for either model.
- The UCR results are one run on the default split, where the bake-off averages 30 resamples.
- The comparison includes XGBoost and ExtraTrees only, not LightGBM or CatBoost, and ExtraTrees' CPU time is not directly comparable.

Future work. The largest datasets suggest that a round budget that grows with the training pool would buy accuracy for minutes of training. A configuration tuned for speed rather than accuracy would map the trade-off on the test folds. Choosing the bit width from the number of rows per class could unify the UCR and MONSTER settings, and cheaper rounds would help on small datasets, where a round's cost is fixed overhead.

7 Conclusion

Gradient boosting does not need a tree, or a search over the training set, to be accurate at scale. A global partition placed between hard examples, with leaves refined by every later mini-batch, matched tuned XGBoost on MONSTER at a cost set by its own budget rather than by the data. That makes it fastest where tree boosting is slowest, and the wrong choice on small datasets, where ExtraTrees is both faster and more accurate.

Acknowledgements

I thank Angus Dempster for supervising this project. This work was supported by the MASSIVE HPC facility (`www.massive.org.au`).

References

[1] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, pp. 3146–3154, 2017.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems 31*, pp. 6638–6648, 2018.

[5] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[6] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems 35: Datasets and Benchmarks Track*, 2022.

[7] A. Dempster, D. F. Schmidt, and G. I. Webb, "QUANT: A minimalist interval method for time series classification," *Data Mining and Knowledge Discovery*, 2024.

[8] N. Cabello and L. Kulik, "Pulsar: Advancing interval-based time series classification to state-of-the-art performance," in *2025 IEEE International Conference on Data Mining (ICDM)*, pp. 1085–1094, 2025.

[9] A. Dempster, N. M. Foumani, C. W. Tan, L. Miller, A. Mishra, M. Salehi, C. Pelletier, D. F. Schmidt, and G. I. Webb, "Monster: Monash scalable time series evaluation repository," 2025.

[10] J. H. Friedman, "Stochastic gradient boosting," *Computational Statistics & Data Analysis*, vol. 38, no. 4, pp. 367–378, 2002.

[11] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[12] S. Popov, S. Morozov, and A. Babenko, "Neural oblivious decision ensembles for deep learning on tabular data," in *International Conference on Learning Representations*, 2020.

[13] Y. Lou and M. Obukhov, "BDT: Gradient boosted decision tables for high accuracy and scoring efficiency," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1893–1901, 2017.

[14] A. Buja, T. Hastie, and R. Tibshirani, "Linear smoothers and additive models," *The Annals of Statistics*, vol. 17, no. 2, pp. 453–510, 1989.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[16] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.

[17] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pp. 604–613, 1998.

[18] R. Mitchell and E. Frank, "Accelerating the XGBoost algorithm using GPU computing," *PeerJ Computer Science*, vol. 3, p. e127, 2017.

[19] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 71–80, 2000.

[20] N. C. Oza and S. Russell, "Online bagging and boosting," in *Proceedings of the Eighth International Workshop on Artificial Intelligence and Statistics*, 2001.

[21] A. Beygelzimer, E. Hazan, S. Kale, and H. Luo, "Online gradient boosting," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[22] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020.

[23] A. Dempster, D. F. Schmidt, and G. I. Webb, "MiniRocket: A very fast (almost) deterministic transform for time series classification," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, (New York), pp. 248–257, ACM, 2021.

[24] A. Dempster, D. F. Schmidt, and G. I. Webb, "Hydra: Competing convolutional kernels for fast and accurate time series classification," *Data Mining and Knowledge Discovery*, vol. 37, pp. 1779–1805, 2023.

[25] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. Bagnall, "HIVE-COTE 2.0: A new meta ensemble for time series classification," *Machine Learning*, vol. 110, no. 11, pp. 3211–3243, 2021.

[26] H. Deng, G. Runger, E. Tuv, and M. Vladimir, "A time series forest for classification and feature extraction," *Information Sciences*, vol. 239, pp. 142–153, 2013.

[27] M. Middlehurst, J. Large, and A. Bagnall, "The canonical interval forest (CIF) classifier for time series classification," in *2020 IEEE International Conference on Big Data (Big Data)*, pp. 188–195, 2020.

[28] N. Cabello, E. Naghizade, J. Qi, and L. Kulik, "Fast, accurate and interpretable time series classification through randomization," 2021.

[29] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The UCR time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[30] M. Middlehurst, P. Schäfer, and A. Bagnall, "Bake off redux: A review and experimental evaluation of recent time series classification algorithms," *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 1958–2031, 2024.

[31] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.